

General Instructions

- Run the original program before making changes.
- Identify bugs systematically before asking AI for a complete solution.
- Use AI to explain causes and suggest fixes.
- Verify every AI suggestion by testing it yourself.
- Record the bug, cause, fix and test result.
- Do not blindly copy AI-generated code.

Q1 – AI-Assisted Bubble Sort Analysis

Tasks: Implement Bubble Sort in ascending order, count comparisons and swaps, explain one complete pass, test at least three input lists, ask about early-exit optimisation, and compare the original and improved versions.

1. Bubble Sort – Original Version

```
numbers = [64, 34, 25, 12, 22, 11, 90]

comparisons = 0
swaps = 0

for i in range(len(numbers)):
    for j in range(len(numbers) - 1 - i):
        comparisons += 1

        if numbers[j] > numbers[j + 1]:
            numbers[j], numbers[j + 1] = numbers[j + 1], numbers[j]
            swaps += 1

print("Sorted list:", numbers)
print("Comparisons:", comparisons)
print("Swaps:", swaps)
```

Output: Sorted list = [11, 12, 22, 25, 34, 64, 90]; Comparisons = 21; Swaps = 14.

2. One Complete Pass Explanation

```
Initial:
[64, 34, 25, 12, 22, 11, 90]

64 > 34 → swap
[34, 64, 25, 12, 22, 11, 90]

64 > 25 → swap
[34, 25, 64, 12, 22, 11, 90]

64 > 12 → swap
[34, 25, 12, 64, 22, 11, 90]

64 > 22 → swap
[34, 25, 12, 22, 64, 11, 90]
```

```
64 > 11 → swap  
[34, 25, 12, 22, 11, 64, 90]
```

```
64 < 90 → no swap
```

After the first pass, the largest element (90) reaches the end of the list.

3. Early-Exit Optimisation

```
numbers = [64, 34, 25, 12, 22, 11, 90]  
  
comparisons = 0  
swaps = 0  
  
for i in range(len(numbers)):  
    swapped = False  
  
    for j in range(len(numbers) - 1 - i):  
        comparisons += 1  
  
        if numbers[j] > numbers[j + 1]:  
            numbers[j], numbers[j + 1] = numbers[j + 1], numbers[j]  
            swaps += 1  
            swapped = True  
  
    if not swapped:  
        break  
  
print("Sorted list:", numbers)  
print("Comparisons:", comparisons)  
print("Swaps:", swaps)
```

The optimisation stops the algorithm when a complete pass makes no swaps. This is useful for already sorted or nearly sorted data.

4. Test Cases

Input	Sorted Output	Comparisons	Swaps
[64,34,25,12,22,11,90]	[11,12,22,25,34,64,90]	21	14
[1,2,3,4,5]	[1,2,3,4,5]	10	0
[5,4,3,2,1]	[1,2,3,4,5]	10	10

Conclusion: Early-exit Bubble Sort avoids unnecessary passes when the list is already sorted or nearly sorted.

Q2 – Linear Search vs Binary Search

Tasks: Implement Linear Search and Binary Search, count comparisons, test first/middle/last/absent elements, explain why Binary Search requires sorted data, compare use cases, and suggest a valid optimisation.

1. Linear Search

```
def linear_search(numbers, target):  
    comparisons = 0  
  
    for i in range(len(numbers)):  
        comparisons += 1  
  
        if numbers[i] == target:
```

```
        return i, comparisons

    return -1, comparisons
```

2. Binary Search

```
def binary_search(numbers, target):
    low = 0
    high = len(numbers) - 1
    comparisons = 0

    while low <= high:
        mid = (low + high) // 2
        comparisons += 1

        if numbers[mid] == target:
            return mid, comparisons

        elif numbers[mid] < target:
            low = mid + 1

        else:
            high = mid - 1

    return -1, comparisons
```

3. Test Program

```
numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]
targets = [5, 31, 89, 100]

for target in targets:
    l_index, l_comparisons = linear_search(numbers, target)
    b_index, b_comparisons = binary_search(numbers, target)

    print("\nTarget:", target)
    print("Linear Search:", l_index)
    print("Linear Comparisons:", l_comparisons)
    print("Binary Search:", b_index)
    print("Binary Comparisons:", b_comparisons)
```

Target	Position	Linear Comparisons	Binary Comparisons
5	0	1	3
31	4	5	1
89	8	9	4
100	Not found	9	4

4. Why Binary Search Needs Sorted Data

Binary Search checks the middle element and eliminates either the left or right half. This decision is valid only when the elements are ordered. Without sorted data, we cannot safely discard half of the list.

5. Complexity and Use Cases

Algorithm	Time Complexity	Suitable Use
Linear Search	$O(n)$	Small or unsorted data
Binary Search	$O(\log n)$	Large, sorted data

Valid optimisation: If the same data will be searched many times, sort it once and then use Binary Search for repeated searches.

Q3 – Compare Sorting Algorithms

Tasks: Implement Bubble Sort, Selection Sort and Insertion Sort using the same input; count comparisons and swaps/shifts; explain strengths and weaknesses; identify the best algorithm for nearly sorted data; and compare experimental results with theoretical complexity.

```
data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]
```

Complete Program

```
data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]
```

```
def bubble_sort(arr):
    arr = arr.copy()
    comparisons = 0
    swaps = 0

    for i in range(len(arr)):
        for j in range(len(arr) - 1 - i):
            comparisons += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swaps += 1

    return arr, comparisons, swaps


def selection_sort(arr):
    arr = arr.copy()
    comparisons = 0
    swaps = 0

    for i in range(len(arr) - 1):
        min_index = i

        for j in range(i + 1, len(arr)):
            comparisons += 1
            if arr[j] < arr[min_index]:
                min_index = j

        if min_index != i:
            arr[i], arr[min_index] = arr[min_index], arr[i]
            swaps += 1

    return arr, comparisons, swaps


def insertion_sort(arr):
    arr = arr.copy()
    comparisons = 0
    shifts = 0

    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0:
            comparisons += 1
```

```

        if arr[j] > key:
            arr[j + 1] = arr[j]
            shifts += 1
            j -= 1
        else:
            break

    arr[j + 1] = key

return arr, comparisons, shifts

print("Original data:", data)

result, comparisons, swaps = bubble_sort(data)
print("\nBubble Sort")
print("Sorted:", result)
print("Comparisons:", comparisons)
print("Swaps:", swaps)

result, comparisons, swaps = selection_sort(data)
print("\nSelection Sort")
print("Sorted:", result)
print("Comparisons:", comparisons)
print("Swaps:", swaps)

result, comparisons, shifts = insertion_sort(data)
print("\nInsertion Sort")
print("Sorted:", result)
print("Comparisons:", comparisons)
print("Shifts:", shifts)

```

All three algorithms produce: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89].

Algorithm	Best	Average	Worst
Bubble Sort	$O(n)$ with early exit	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$

Strengths/weaknesses: Bubble Sort is simple but usually slow; Selection Sort performs few swaps but still takes $O(n^2)$ comparisons; Insertion Sort is simple and performs especially well on nearly sorted data.

Best for nearly sorted data: Insertion Sort.

Q4 – Optimising a Student Search and Sort Program

Tasks: Write functions for highest mark, sorting and name search; create a straightforward solution; identify unnecessary operations; improve it using a technique you understand; and compare time complexity.

```

students = [
    ("Arun", 82),
    ("Bala", 65),
    ("Chitra", 91),
    ("Divya", 74),
    ("Esha", 88)
]

```

1. Straightforward Solution

```
students = [
    ("Arun", 82),
    ("Bala", 65),
    ("Chitra", 91),
    ("Divya", 74),
    ("Esha", 88)
]

def highest_mark(students):
    highest = students[0]

    for student in students:
        if student[1] > highest[1]:
            highest = student

    return highest

def sort_students(students):
    return sorted(students, key=lambda x: x[1], reverse=True)

def search_student(students, name):
    for student in students:
        if student[0].lower() == name.lower():
            return student

    return None

print("Highest mark:", highest_mark(students))

print("\nStudents sorted by marks:")
for student in sort_students(students):
    print(student)

print("\nSearch result:")
print(search_student(students, "Chitra"))
```

Expected result: Highest mark = Chitra (91). Sorted order = Chitra, Esha, Arun, Divya, Bala. Searching for Chitra returns ('Chitra', 91).

2. Improved Approach Using a Dictionary

```
students = [
    ("Arun", 82),
    ("Bala", 65),
    ("Chitra", 91),
    ("Divya", 74),
    ("Esha", 88)
]

student_dict = {}

for name, mark in students:
    student_dict[name] = mark
```

```

highest = max(students, key=lambda x: x[1])

sorted_students = sorted(
    students,
    key=lambda x: x[1],
    reverse=True
)

name = "Chitra"

if name in student_dict:
    print(name, "has", student_dict[name], "marks")
else:
    print("Student not found")

print("Highest:", highest)

print("Sorted students:")
for student in sorted_students:
    print(student)

```

Operation	Simple Approach	Improved Approach
Highest mark	$O(n)$	$O(n)$
Sorting	$O(n \log n)$	$O(n \log n)$
Name search	$O(n)$	$O(1)$ average
Extra space	$O(1)$	$O(n)$

Conclusion: The dictionary uses additional memory, but repeated name searches become much faster on average.

Q5 – Optimisation Challenge: Best Combination

This problem asks for a combination of items whose total price is as high as possible without exceeding the budget. The formulation is a 0/1 knapsack problem because each item can be selected at most once.

```

items = [
    ("Laptop", 60000),
    ("Phone", 30000),
    ("Tablet", 20000),
    ("Watch", 10000),
    ("Headphones", 5000)
]

budget = 50000

```

1. Brute Force Solution

```

from itertools import combinations

items = [
    ("Laptop", 60000),
    ("Phone", 30000),
    ("Tablet", 20000),
    ("Watch", 10000),
    ("Headphones", 5000)
]

budget = 50000

```

```

best_items = []
best_total = 0

for r in range(len(items) + 1):
    for combination in combinations(items, r):
        total = sum(price for name, price in combination)

        if total <= budget and total > best_total:
            best_total = total
            best_items = combination

print("Best combination:")
for item in best_items:
    print(item)

print("Total:", best_total)

```

For a budget of ₹50,000, the best combination is Phone + Tablet = ₹50,000.

2. Greedy Approach – Important Note

A greedy method may work for the given input, but greedy selection does not guarantee an optimal answer for every 0/1 knapsack problem. Therefore, it should not be claimed as the guaranteed solution for this formulation.

3. Dynamic Programming – Recommended Solution

```

items = [
    ("Laptop", 60000),
    ("Phone", 30000),
    ("Tablet", 20000),
    ("Watch", 10000),
    ("Headphones", 5000)
]

def knapsack(items, budget):
    n = len(items)

    dp = [[0] * (budget + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        name, price = items[i - 1]

        for b in range(budget + 1):

            if price <= b:
                dp[i][b] = max(
                    dp[i - 1][b],
                    price + dp[i - 1][b - price]
                )
            else:
                dp[i][b] = dp[i - 1][b]

    selected = []
    b = budget

    for i in range(n, 0, -1):

```



```

        if dp[i][b] != dp[i - 1][b]:
            selected.append(items[i - 1])
            b -= items[i - 1][1]

    selected.reverse()

    return selected, dp[n][budget]

selected, total = knapsack(items, 50000)

print("Selected items:")
for item in selected:
    print(item)

```

```
print("Maximum total:", total)
```

Expected result for ₹50,000: Phone (₹30,000) + Tablet (₹20,000) = ₹50,000.

Dynamic Programming guarantees the optimal solution for the 0/1 knapsack formulation because it considers the best achievable value for each item/budget subproblem.

4. Test with Three Budgets

```

budgets = [25000, 50000, 75000]

for budget in budgets:
    selected, total = knapsack(items, budget)

    print("\nBudget:", budget)
    print("Selected items:")

    for item in selected:
        print(item)

    print("Maximum total:", total)

```

Budget	One Optimal Combination	Maximum Total
₹25,000	Tablet + Headphones	₹25,000
₹50,000	Phone + Tablet	₹50,000
₹75,000	Phone + Tablet + Watch + Headphones	₹65,000

5. Complexity and Optimisation

The two-dimensional Dynamic Programming solution has time complexity $O(n \times \text{budget})$ and space complexity $O(n \times \text{budget})$. For larger inputs, the DP table can be reduced to a one-dimensional array, reducing space to $O(\text{budget})$.

Final Submission Checklist

- Run every program yourself and verify the output.
- Take screenshots of the code/output if your teacher requires them.
- For each question, include the explanation, test cases and conclusion.
- Record any bug found, its cause, the fix and the test result.
- Do not submit AI-generated code without understanding and testing it.

- For Q5, clearly state that Dynamic Programming guarantees the optimal solution for the 0/1 knapsack formulation.